Labo 10 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — 40 Go dans `userdata/`

Réponse. C'est le **fichier de logs** du conteneur (`ctr.log`, pilote `k8s-file`) : une application bavarde qui tourne depuis des semaines, sans rotation. `podman system df` compte les images, les couches writable et les volumes, pas les logs : ils ne sont ni « récupérables » ni « supprimables » à ses yeux. Correction immédiate : recréer le conteneur (les logs partent avec lui) ou tronquer le fichier (`: > ctr.log`, acceptable en urgence). Correction durable : borner la taille (`--log-opt max-size=10m`, ou `log_size_max` dans `containers.conf`), expédier les logs vers une plateforme centrale, et réduire le niveau de log de l'application.

Pourquoi. Le moteur capture stdout dans un fichier qui n'a pas de limite par défaut ; seule une rotation ou une taille maximale l'arrête.

Nuance. Avec le pilote `journald` (par défaut sur une Ubuntu avec systemd), c'est le journal système qui grossit, borné par sa propre configuration (`SystemMaxUse`).

Exemple.

```
podman inspect api --format '{{.HostConfig.LogConfig.Type}} {{.HostConfig.LogConfig.Path}}'
du -sh $(podman inspect api --format '{{.HostConfig.LogConfig.Path}}')
```

Question 2 — `Exited (137)` sans `OutOfMemoryError`

Réponse. Avec `MaxRAMPercentage=90`, le tas Java peut atteindre 230 Mo sur une limite de 256 Mo ; le reste de la JVM (metaspace, threads, code JIT, tampons) dépasse les 26 Mo restants. La mémoire **totale** du cgroup atteint `memory.max`, le noyau déclenche l'OOM killer et tue `java` avec `SIGKILL` : code `137`, aucune exception Java (la JVM n'a jamais manqué de tas, c'est le conteneur qui a manqué de mémoire). Confirmation : `podman inspect api --format '{{.State.OOMKilled}}'` → `true`, et `podman events --filter event=oom`. Corrections : baisser `MaxRAMPercentage` à 50-75 %, ou monter `--memory`.

Pourquoi. La limite cgroup s'applique à **tout** le processus ; le tas n'en est qu'une partie. `OutOfMemoryError` ne survient que si le **tas** est plein, ce qui n'arrive jamais quand le noyau frappe avant.

Nuance. `-XX:+ExitOnOutOfMemoryError` et un `--restart` masquent le symptôme sans le corriger. Dimensionner, c'est mesurer : `podman stats` pendant un test de charge.

Exemple.

```
podman inspect api --format 'oom={{.State.OOMKilled}} exit={{.State.ExitCode}}'   # oom=true exit=137
podman exec api sh -c 'java -XX:MaxRAMPercentage=90 -XX:+PrintFlagsFinal -version 2>/dev/null | grep " MaxHeapSize"'
```

Question 3 — `stats`, `top`, `inspect`

Réponse. `podman stats` lit les **compteurs cgroup** en temps réel : CPU %, mémoire utilisée / limite, réseau, I/O. `podman top` liste les **processus** du conteneur (UID, PID, commande), comme `ps` vu de l'intérieur. `podman inspect` décrit la **configuration et l'état** : ce qui a été demandé (limites, variables, montages), pas ce qui est consommé. Sans `--memory`, le cgroup n'a pas de plafond (`memory.max = max`) et `stats` affiche la mémoire **de la machine** comme limite : 32 Go ne signifie pas « réservé », mais « rien n'arrête ce conteneur avant ».

Pourquoi. Trois sources : le noyau (cgroups, `/proc`) pour `stats` et `top`, la base de données de Podman pour `inspect`.

Nuance. `stats` en rootless exige la délégation cgroup ; `top` accepte des colonnes (`podman top api user pid args`) mais pas toute la syntaxe de `ps`.

Exemple.

```
podman stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'
podman inspect api --format 'mem={{.HostConfig.Memory}}'   # 0 = pas de limite
```

Question 4 — Le `HEALTHCHECK` disparu

Réponse. Buildah construit par défaut une image au format **OCI**, dont la spécification n'a pas de champ pour le healthcheck : l'instruction est ignorée, avec un seul avertissement perdu dans la sortie du build (`HEALTHCHECK is not supported for OCI image format and will be ignored`). Vérification : `podman image inspect api:1.1 --format '{{.Config.Healthcheck}}'` → `<nil>`. Correction : `podman build --format docker -t api:1.1 .`, après quoi le même inspect montre `[CMD-SHELL wget …]` et `podman ps` affiche `(healthy)`. Docker construit au format Docker par défaut : le healthcheck y est toujours conservé.

Pourquoi. `HEALTHCHECK` est une extension Docker du format d'image, jamais reprise dans la norme OCI (qui laisse ce rôle à l'orchestrateur).

Nuance. Une image OCI reste la bonne cible pour Kubernetes, qui ignore de toute façon le `HEALTHCHECK` et utilise ses propres sondes. Le format docker ne compte que pour Podman/Docker/Compose.

Exemple.

```
podman build --format docker -q -t api:1.1 . && podman image inspect api:1.1 --format
'{{.Config.Healthcheck.Test}}'
```

Question 5 — `unhealthy`, et puis ?

Réponse. Avec Podman seul : **rien**. L'état passe à `unhealthy` dans `podman ps`, un événement `health_status` est émis, le conteneur continue de tourner et de répondre mal. Docker fait pareil : le healthcheck est un **indicateur**, pas un correcteur ; seuls Swarm ou Compose avec `depends_on` s'en servent. Podman ajoute `--health-on-failure=kill|restart|stop` : avec `kill` et `--restart=always`, le conteneur est tué puis relancé. L'orchestrateur reste la vraie réponse parce qu'il combine la sonde avec le **retrait du trafic** (readiness), le **remplacement** (liveness) et le **report ailleurs** si la machine est en cause.

Pourquoi. Décider de tuer un service est une décision de pilotage, que le moteur de conteneurs laisse volontairement à une couche supérieure.

Nuance. Une liveness probe mal réglée tue en boucle un service lent : la sonde doit tester la **vivacité**, pas la charge.

Exemple.

```
podman run -d --name api --health-on-failure=kill --restart=always api:1.1
podman events --stream=false --since 10m --filter event=health_status | tail -2
```

Question 6 — Le conteneur durci

Réponse. `uid=1000(…) gid=1000(…)` (l'image a `USER 1000:1000`, `--user 1000` le confirme) ; `touch: /app/x: Read-only file system` ; `touch: /tmp/x: Read-only file system` **aussi**, car aucun `--tmpfs /tmp` n'a été donné ; `java -version` fonctionne (lire ne demande rien). Pour une vraie API Spring Boot : ajouter `--tmpfs /tmp` (Tomcat y crée son répertoire de travail, la JVM y écrit `hsperfdata`), et si l'application écrit des fichiers (uploads, cache), un volume dédié. Rien d'autre : une API n'a besoin d'aucune capability.

Pourquoi. `--read-only` verrouille toute l'image, y compris `/tmp` ; les écritures légitimes doivent avoir un montage explicite.

Nuance. `--cap-drop ALL` avec `--user 1000` est presque redondant (un non-root n'a déjà pas les capabilities), mais protège contre un binaire setuid dans l'image — ce que `no-new-privileges` verrouille aussi.

Exemple.

```
podman run --rm --read-only --tmpfs /tmp --cap-drop ALL --user 1000 --security-opt no-new-
privileges --entrypoint sh api:1.0 -c 'touch /tmp/x && echo OK'
```

Question 7 — Root sans pouvoirs

Réponse. Dans le premier cas, root dispose des capabilities par défaut du conteneur, dont `CAP_CHOWN` ; `chown` réussit. Avec `--cap-drop ALL`, le processus est toujours UID 0, mais sans **aucune** capability : le noyau refuse `chown` (`Operation not permitted`), comme il le refuserait à un utilisateur ordinaire. L'exemple montre que, sous Linux, « root » n'est qu'un numéro ; les **privilèges** sont des capabilities attachées au processus, que l'on peut retirer une à une. Un attaquant qui devient root dans un conteneur `--cap-drop ALL` n'a que les droits d'un utilisateur sans pouvoirs sur un système de fichiers qu'il ne peut pas modifier.

Pourquoi. Depuis Linux 2.2, les vérifications de privilèges du noyau testent des capabilities, pas l'UID ; UID 0 n'est qu'une façon d'en obtenir le jeu complet.

Nuance. En rootless, les capabilities du conteneur ne valent que dans le **user namespace** : même `CAP_SYS_ADMIN` n'y donne aucun pouvoir sur l'hôte. Deux couches de protection.

Exemple.

```
podman run --rm --entrypoint sh alpine -c 'chown nobody /etc/hostname && echo OK'          # OK
podman run --rm --cap-drop ALL --entrypoint sh alpine -c 'chown nobody /etc/hostname'      #
Operation not permitted
```

Question 8 — `latest` en production

Réponse. Trois incidents : (1) **deux serveurs, deux versions** — un `pull` à 10 h, un autre à 10 h 05 après un nouveau build, et personne ne sait lequel tourne où ; (2) **retour arrière impossible** — `latest` a été écrasé, l'image précédente n'a plus de nom ; (3) **déploiement non reproductible** — un `pull` en recette et un en production ne donnent pas le même contenu, le test ne prouve rien. Stratégie : `api:1.4.2` + `api:1.4` + `api:1` + `api:git-<sha>`, jamais `latest` déployé. Le déploiement **par digest** garantit en plus que même un tag de version **réécrit** (par erreur, ou par un attaquant du registry) ne change pas ce qui tourne : le digest est l'empreinte du contenu, un tag n'est qu'un nom.

Pourquoi. Un tag est un pointeur mobile stocké côté registry ; un digest est calculé à partir des couches.

Nuance. Les tags glissants (`1.4`, `1`) sont utiles pour les **consommateurs** (une image de base), pas pour le déploiement de **votre** application.

Exemple.

```
podman image inspect api:1.0 --format '{{.Digest}}'
podman pull localhost:5000/api@sha256:52b82ed9…   # immuable, quoi qu'il arrive au tag
```

Question 9 — HTTPS attendu

Réponse. Podman parle **TLS** par défaut à tout registry, y compris `localhost:5000` ; `registry:2` sans certificat répond en HTTP clair, d'où `server gave HTTP response to HTTPS client`. Test local : `--tls-verify=false` sur `push` et `pull`. Entreprise : un **certificat** valide sur le registry (et la CA dans `/etc/containers/certs.d/<registry>/ca.crt` si elle est interne). Entre les deux, `[[registry]] location="…" insecure=true` dans `registries.conf` déclare l'exception une fois pour toutes. La première ne doit jamais devenir la seconde parce qu'un registry en clair se laisse **intercepter et remplacer** : une image substituée sur le chemin réseau est une compromission de toute la chaîne de déploiement.

Pourquoi. Le registry distribue du code exécutable ; TLS garantit son origine et son intégrité en transit.

Nuance. Docker refuse de la même façon et se configure par `insecure-registries` dans `daemon.json` — un réglage du daemon, donc de l'administrateur.

Exemple.

```
podman push --tls-verify=false localhost:5000/api:1.0 && curl -s localhost:5000/v2/_catalog
```

Question 10 — Même `IMAGE ID` après l'aller-retour

Réponse. Le registry a reçu les **couches** (blobs), la **configuration** et le **manifeste** de l'image, chacun identifié par son hachage ; au `pull`, Podman a vérifié chaque hachage et reconstitué exactement la même image. L'`IMAGE ID` est le hachage de la configuration : contenu identique, ID identique — le nom `localhost:5000/api:1.0` n'est qu'un tag de plus sur la même image locale. `--format '{{.Digest}}'` donne le hachage du **manifeste**, celui que le registry utilise : c'est lui qu'on référence pour un déploiement immuable, et lui qui change si une couche change.

Pourquoi. Le stockage est **adressé par contenu** : deux chemins qui mènent au même contenu mènent au même objet.

Nuance. Un même contenu poussé vers deux registries peut avoir deux digests de manifeste si le format diffère (OCI contre Docker) ; l'ID de configuration, lui, reste identique.

Exemple.

```
podman images --format '{{.Repository}}:{{.Tag}} {{.ID}}' | grep api     # deux noms, un ID
curl -s localhost:5000/v2/api/manifests/1.0 -H 'Accept:
application/vnd.oci.image.manifest.v1+json' | head -c 200
```

Question 11 — `system prune -a --volumes -f` en pleine CI

Réponse. Supprimé : tous les conteneurs **arrêtés**, les réseaux inutilisés, **toutes** les images sans conteneur en cours (`-a` : images de base, `api:1.0`, cache de build), et tous les volumes qu'aucun conteneur **existant** n'utilise. Non supprimé : les conteneurs en cours d'exécution, leurs images et leurs volumes — la stack de tests survit… jusqu'à son `down`, après quoi ses volumes seront orphelins. Build suivant : retéléchargement des images de base (minutes, quota Docker Hub), reconstruction sans cache, et perte des volumes de données partagés entre jobs. Séquence correcte : `podman system df`, `podman image prune` (dangling seulement), `podman container prune` si les conteneurs arrêtés sont connus, suppression **nommée** des volumes après lecture de `volume ls --filter dangling=true`.

Pourquoi. `prune` raisonne en « référencé par un conteneur existant », pas en « utile demain ».

Nuance. Sur un agent de CI éphémère (recréé à chaque job), `prune -a` est au contraire la norme ; le problème vient de l'agent **persistant** où le cache est précieux.

Exemple.

```
podman system df && podman volume ls --filter dangling=true && podman image prune -f
```

Question 12 — Le Quadlet

Réponse. `daemon-reload` fait lire le fichier `api.container` par le générateur Quadlet, qui produit une unité `api.service` avec un `ExecStart=podman run … localhost/api:1.0` et les options traduites (`-p 18080:8080`, `--env-file`). `enable --now` démarre le conteneur et l'inscrit dans `default.target` : à chaque démarrage de la session utilisateur, systemd relance `podman run`. `--restart=always` échoue après `wsl --shutdown` parce qu'aucun processus Podman ne survit pour l'appliquer ; ici, c'est **systemd** qui est relancé par le système, et qui relance Podman. Sur WSL, il faut `loginctl enable-linger $USER` pour que la session `--user` existe sans connexion, et que la distribution soit démarrée (un `wsl` ouvert ou une tâche planifiée).

Pourquoi. Un service systemd a un superviseur permanent ; un conteneur `--restart` n'en a pas sans daemon.

Nuance. Quadlet génère des conteneurs **jetables** (`podman run --rm --replace`) : les données doivent être dans des volumes, idéalement déclarés par un `.volume` Quadlet.

Exemple.

```
systemctl --user daemon-reload && systemctl --user enable --now api && systemctl --user status
api --no-pager | head -3
journalctl --user -u api -n 5 --no-pager
```

Question 13 — Le pipeline

Réponse. (1) **Build** — `podman build --format docker -t api:1.4.2 -t api:git-a1b2c3d .` (labos 04-05) ; échec si le Dockerfile ou les tests du stage `test` échouent. (2) **Scan** — Trivy/Grype sur l'image (labo 05) ; échec sur CVE critique. (3) **Push** — `podman push` vers le registry privé, avec TLS (labos 02, 10) ; échec si le registry refuse (droits, quota). (4) **Recette** — `podman compose -f compose.yaml -f compose.staging.yaml up -d` sur l'image **par digest**, tests d'intégration, `down -v` (labo 09) ; échec si un test ou un healthcheck ne passe pas. (5) **Production** — l'orchestrateur déploie le **même digest** (labos 08, 11+), avec les limites, le `securityContext` et les sondes écrites dans les manifestes. Le scan se place avant le push (on ne distribue pas une image vulnérable) ; le `HEALTHCHECK` est dans l'image dès le build ; les limites de ressources sont dans les fichiers de déploiement (Compose ou manifestes), pas dans l'image ; le digest est calculé au build, enregistré au push, et référencé à partir de la recette.

Pourquoi. Chaque étape produit un artefact vérifiable et immuable pour la suivante : image, rapport de scan, digest, résultat de tests.

Nuance. Le pipeline est le même avec Docker ; seules les commandes de build/push et le format d'image changent.

Exemple.

```
DIGEST=$(podman image inspect api:1.4.2 --format '{{.Digest}}') && echo "deploy:
registry.example.com/api@$DIGEST"
```